MineShield

Mineral Transportation Record Reconciliation & Compliance Audit Platform

Target Domain: RegTech & Mineral Supply Chain Audit	**Architecture:** React 19 (Vite) + Node.js (Express 5)
Database: 3NF Relational (SQLite / Azure SQL)	**Automated Verification:** 24 Integration Tests Passed

1. Executive Summary & Problem Scope

In mineral extraction (sand, quartzite, stone aggregate, bauxite), illegal mineral diversion and revenue leakage occur because operational records are kept in organizational silos. Departmental mining portals issue e-permits, highway checkpoints/tollgates observe passing vehicles, and destination weighbridges weigh actual net delivery tonnage. Due to the lack of integrated auditing, discrepancies—such as overloaded trucks, repeated trips on single permits, and corridor bypassing—routinely go undetected.

Core Scope Safeguard: MineShield is an operational audit-support platform. It automates multi-source ingestion, deterministic record matching, discrepancy categorization, and evidence management. It surfaces evidence-backed anomalies for human review but does *not* make unsupported legal conclusions, replace legal mining portals, or automatically levy fines.

2. System Architecture & Tech Stack

MineShield employs a decoupled, modular multi-tier architecture with dual-provider abstraction for enterprise cloud scalability:

Tier	Technology	Key Responsibilities & Protections
Frontend SPA	React 19, Vite 8	11 primary views, CSV preview with header mapping, side-by-side record comparison, case modals, responsive dashboard.
Backend API	Node.js v24, Express 5	Deterministic reconciliation engine, JWT validation, in-memory rate limiter (HTTP 429), binary magic-byte validator.
Relational DB	SQLite (Local) / Azure SQL DB (Cloud)	3NF normalized schema (15 tables), foreign key cascades, ACID transactions via BEGIN IMMEDIATE, 100% parameterized SQL.
Storage Layer	./uploads (Local) / Azure Blob Storage	Evidence slips and document persistence, SHA-256 hash recalculation, on-demand tamper verification.

3. The Eight Automated Reconciliation Rules

Rule	Category Code	Detection Logic & Purpose	Severity
Rule 1	QUANTITY_MISMATCH	Permit tonnage diverges from weighbridge scale net weight beyond configurable tolerance (default: 5.0%).	MED / HIGH
Rule 2	MISSING_CHECKPOINT	Weighbridge delivery recorded, but vehicle never observed at highway checkpoint. Evaluated only when checkpoint source is imported.	HIGH
Rule 3	MISSING_WEIGHBRIDGE	Vehicle observed passing transit checkpoint, but no scale delivery slip registered at destination weighbridge.	HIGH
Rule 4	VEHICLE_MISMATCH	Permit authorizes vehicle 'A', but checkpoint RFID or weighbridge logged vehicle 'B' for that permit number.	HIGH
Rule 5	DUPLICATE_RECORD	Same weighbridge transaction ID or scale slip appears multiple times in the imported operational dataset.	MEDIUM
Rule 6	INCOMPLETE_RECORD	Checkpoint surveillance observation is missing a valid transit permit reference number.	LOW
Rule 7	AMBIGUOUS_MATCH	Vehicle without permit reference links to multiple active candidate permits during the transit window.	MEDIUM
Rule 8	CROSS_SOURCE_INCONSISTENCY	Direct contradictions: checkpoint weight diverges from scale net weight > tolerance, or checkpoint transit timestamp is AFTER weighbridge timestamp.	MED / HIGH

4. Team Member Roles & Responsibilities

For final academic presentation and evaluation, project responsibilities are divided across four specialized engineering domains:

Role / Specialization	Key Responsibilities	Deliverables in Codebase
Role 1: Team Lead & Reconciliation Engine Architect	Core system architecture, deterministic matching logic, reconciliation tolerance boundary algorithms, idempotency, dynamic changed-data auto-resolution.	services/reconciliationEngine.js server.js, config.js
Role 2: Database Architect & Cloud Integration Lead	3NF relational schema, foreign key cascades, ACID transaction boundaries (BEGIN IMMEDIATE / ROLLBACK), dual-provider Azure SQL / SQLite abstraction.	db/schema.sql, db/index.js db/schema.azure.sql db/azureSqlProvider.js
Role 3: Frontend & UI/UX Engineer	React SPA construction (11 views), CSV preview modal, column auto-mapping, side-by-side discrepancy investigation UI, case modals, responsive CSS.	mineshield-frontend/src/App.jsx App.css, index.html
Role 4: Security & Quality Assurance Auditor	JWT RBAC, binary magic-byte content validation, CSV formula injection defense, rate limiting, SHA-256 evidence integrity, 24 automated integration tests.	utils/fileValidation.js utils/csvSanitizer.js middleware/rateLimiter.js test.js (24 tests)

5. Comprehensive Viva Voce Questions & Model Answers

Prepare for external examination and defense with the following technical questions and answers:

Q1: What real-world problem does MineShield solve?

Answer: MineShield addresses mineral revenue leakage and illegal mining diversion. Mining permits, checkpoint RFID scans, and destination weighbridge scale slips exist in disconnected organizational silos. Operators exploit these silos through overloading, multi-tripping on single permits, or bypassing transit corridors. MineShield provides deterministic, automated cross-reconciliation across all three datasets.

Q2: How does the matching algorithm handle records without permit numbers?

Answer: The engine uses a tiered matching fallback. When permit numbers are present, it links records with 1.0 exact confidence. If a checkpoint lacks a permit reference, it falls back to normalized vehicle registration plates (stripped of spaces and hyphens). If exactly one active permit matches that vehicle, it flags a PROBABLE match; if multiple active permits match, it flags AMBIGUOUS_MATCH (Rule 7) for human review.

Q3: What prevents false 'Missing Checkpoint' flags when checkpoint data is unavailable?

Answer: We hardened the engine with source availability inspection. Before evaluating Rule 2 (MISSING_CHECKPOINT) or Rule 3 (MISSING_WEIGHBRIDGE), the engine verifies whether source records exist (checkpoints.length > 0). If the dataset was never imported, the rule is safely skipped and reported in run metadata as checkpointDatasetAvailable: false.

Q4: What is 'Reconciliation Idempotency', and why is it important?

Answer: Idempotency ensures that running reconciliation multiple times on the same dataset does not inflate discrepancy counts or create duplicate open anomalies. Before creating a discrepancy record, the engine checks for an existing active record matching the rule, category, and source IDs. If one exists, duplicate insertion is suppressed.

Q5: How does the application handle data that changes between reconciliation runs?

Answer: When a missing checkpoint or weighbridge record is imported in a subsequent batch, re-running reconciliation detects the newly matched link. The engine automatically transitions the earlier MISSING_CHECKPOINT discrepancy to RESOLVED, timestamps it, and logs an automated audit note: 'Auto-resolved by [RunCode]: Matching record has been imported.'

Q6: Why is your database schema in Third Normal Form (3NF)?

Answer: It satisfies 3NF because: (1) All column attributes are atomic (1NF); (2) All non-key fields depend on the whole primary key (2NF); (3) There are no transitive dependencies (3NF). Batch import summaries live in DataImports, individual row errors live in ImportRowErrors, and the M:N relationship between ComplianceCases and Discrepancies is isolated in CaseDiscrepancies.

Q7: How do you guarantee transaction atomicity during batch imports?

Answer: All multi-step insertions are wrapped in an atomic transaction(callback) handler. In SQLite, it executes BEGIN IMMEDIATE and issues all inserts. If any row or foreign key fails, it executes ROLLBACK and rethrows the error, guaranteeing zero partial or orphan records persist in the database.

Q8: How does your Role-Based Access Control (RBAC) prevent privilege escalation?

Answer: Authorization is enforced strictly on the server using cryptographically signed JWT tokens (HMAC-SHA256). The requireRole middleware decodes the verified token payload. Even if a user alters client state or localStorage, requests to administrative routes (/api/users, /api/settings) are rejected with HTTP 403 Forbidden because client claims are untrusted.

Q9: What is 'Spreadsheet Formula Injection' (CSV Injection), and how did you prevent it?

Answer: CSV Injection occurs when untrusted text fields begin with '=', '+', '-', '@', '\t', or '\r'. When opened in Excel, the software may interpret the cell as a dynamic formula or execute DDE commands. We built utils/csvSanitizer.js, which inspects all exported cells and prepends a single apostrophe (') to force spreadsheet software to treat the value as literal text.

Q10: How do you validate evidence files beyond checking the filename extension?

Answer: We implemented binary magic-byte content inspection in utils/fileValidation.js. Even if an executable is renamed to 'slip.pdf', the server inspects the initial buffer bytes: PDF files must start with %PDF (0x25, 0x50, 0x44, 0x46), PNG files with 0x89504E47, and JPEG files with 0xFFD8FF. Spoofed files are rejected with HTTP 400.

Q11: Does a SHA-256 hash prove that an evidence document is legally authentic?

Answer: No. Cryptographic SHA-256 hashing verifies file integrity, not real-world authenticity. It mathematically proves that the file currently stored on disk has not been altered, corrupted, or replaced since upload. It does not certify that the underlying physical document was genuine.

Q12: How are brute-force attacks mitigated on the authentication endpoint?

Answer: We implemented a sliding-window rate limiter in middleware/rateLimiter.js. It tracks failed login attempts per client IP. If an IP accumulates 10 failed login attempts within a 15-minute window, subsequent attempts are blocked with HTTP 429 Too Many Requests until the lockout timer expires.

Q13: How is the audit trail protected from unauthorized modification or deletion?

Answer: The audit trail is implemented as an application-level append-only ledger. There are no PUT, PATCH, or DELETE routes exposed on /api/audit-logs. Automated tests verify that attempting to mutate or delete an audit record returns HTTP 404 Route Not Found.

Q14: How does the system switch between SQLite and Azure SQL Database?

Answer: We implemented a clean dual-provider database abstraction in db/index.js. When DATABASE_PROVIDER=azuresql, it loads db/azureSqlProvider.js, which manages an mssql connection pool, converts positional '?' parameters to T-SQL '@p1, @p2', and executes against Azure SQL. If Azure credentials are unset, it fails fast with an explicit error rather than silently falling back.

Q15: How many automated tests exist in the test suite, and what do they verify?

Answer: The test suite in test.js comprises 24 comprehensive integration tests covering health checks, JWT RBAC enforcement (401/403), transaction rollback on failure, CSV preview and column mapping, unavailable source isolation, all 8 reconciliation rules, idempotency, changed-data auto-resolution, evidence on-disk SHA-256 verification, magic-byte inspection, CSV formula injection defense, Azure fail-fast configuration, and rate limiting.

6. Quick Operational Commands

Task	Terminal Command
Run 24 Integration Tests	npm.cmd test --prefix C:\Users\HP\Desktop\Coding\mineshield-backend
Pristine Demonstration Seeding	node C:\Users\HP\Desktop\Coding\mineshield-backend\seed-sample.js --clean
Start Backend API Server	npm.cmd start --prefix C:\Users\HP\Desktop\Coding\mineshield-backend (Port 5000)
Start Frontend Application	npm.cmd run dev --prefix C:\Users\HP\Desktop\Coding\mineshield-frontend (Port 5173)